ReLU and Its Improvements in Machine Learning

Name: Dnyaneshwari Maradkar

Student No: 24142497

GitHub Repository: https://github.com/dnyaneshwari7/machine-learning-relu-improvements

1. Introduction

When I first started learning about neural networks, one thing that seemed confusing was how a model made up of mathematical operations could learn something as complex as recognising objects, understanding patterns, or making useful predictions from data. What makes this possible is the activation function. Activation functions introduce non-linearity into a neural network, which means the network can move beyond simple straight-line relationships and learn more realistic patterns from data. Without an activation function, even a deep network with many layers would behave like a basic linear model.

Among the many activation functions used in machine learning, the Rectified Linear Unit, usually called ReLU, is one of the most widely used. It became popular because it is easy to implement, computationally efficient, and effective in practice. In many deep-learning models, ReLU trains faster and more reliably than older activation functions such as sigmoid and tanh. However, even though ReLU is useful, it is not perfect. It has a well-known weakness called the dying ReLU problem, where some neurons can stop learning completely. Because of this limitation, improved versions of ReLU were developed.

This report explains the ReLU activation function, why it is useful, where it falls short, and how improved activation functions such as Leaky ReLU, PReLU, ELU, GELU, and Swish attempt to solve its weaknesses. The goal is not only to describe the formulas, but also to explain in simple terms why small mathematical changes can make a big difference to how a neural network learns.

2. The ReLU Activation Function

The ReLU activation function is defined as follows:

$$f(x) = max(0, x)$$

This can also be written in piecewise form:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

This means that ReLU keeps any positive input unchanged, but turns every negative input into zero. The reason this function is so popular is that it is very simple compared with older activation functions. For

example, sigmoid and tanh require exponential calculations, while ReLU only needs to compare the input with zero. That makes it faster to compute, especially when the activation function is being applied millions of times during training.

The derivative of ReLU is:

$$f(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

At $x = 0$, the derivative is not strictly defined, but in practical implementations this is usually handled without difficulty. The important point is that for positive inputs the gradient remains equal to 1. This helps information and gradients pass through the network during backpropagation, which is one of the main reasons ReLU became the default activation function in many deep-learning models.

3. Why ReLU Works So Well

ReLU became popular in deep learning not just because it is simple, but because it solves several practical problems that earlier activation functions struggled with. One thing I noticed while studying it is that ReLU does not compress positive values into a small range like sigmoid or tanh. Instead, it allows positive inputs to pass through directly. This makes a big difference during training because useful information is not lost as it moves through layers.

Another important reason is how ReLU behaves during backpropagation. For positive inputs, the gradient remains constant, which helps the model keep learning even in deeper networks. When I compared this to sigmoid, it became clearer why sigmoid struggles — its gradients become very small, while ReLU keeps them stable.

ReLU also creates what is called sparse activation, where some neurons output zero. At first, this seemed like a disadvantage to me, but it actually helps the model focus only on important features. In a way, it makes the network more efficient by not activating every neuron for every input.

Overall, what stood out to me is that ReLU is not just mathematically simple — it is practically effective. Its behaviour directly supports faster and more stable learning, which explains why it became the default activation function in many deep learning models.

4. Limitations of ReLU

4.1 The Dying ReLU Problem

The biggest weakness of ReLU is the dying ReLU problem. Since the function outputs zero for all negative inputs, the gradient in that region is also zero. If a neuron keeps receiving negative inputs during training,

its weights may stop updating because there is no gradient signal to push learning forward. When this happens, the neuron can effectively become inactive, or "dead."

A few dead neurons may not destroy the entire network, but if many neurons stop learning, the model loses some of its ability to represent complex patterns. In other words, the network still works, but not as well as it could. This is why researchers started looking for activation functions that keep the advantages of ReLU while avoiding a completely flat negative region.

4.2 Other Limitations

ReLU also has a few other limitations. Its outputs are not zero-centred, since they are always zero or positive. This can make optimisation slightly less balanced. In addition, ReLU is unbounded on the positive side, which means large inputs can produce very large outputs. Finally, ReLU has a sharp corner at zero, so it is not fully smooth. Although this is usually manageable in practice, smoother activation functions can sometimes improve optimisation.

5. Improved Activation Functions

Although ReLU works well, its biggest weakness is the dying ReLU problem, where neurons stop learning if they only receive negative inputs. Because of this, several improved activation functions have been developed. What I found interesting is that each improvement changes only a small part of the formula, but the effect on learning can be quite significant.

5.1 Leaky ReLU

Leaky ReLU modifies ReLU by allowing a small slope for negative inputs instead of forcing them to zero:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

where α is usually a small value like 0.01.

What I found useful about Leaky ReLU is how simple the fix is. Instead of completely shutting off the neuron, it allows a small gradient to pass through. This makes intuitive sense because the neuron is still “alive,” even for negative inputs. However, the slope is fixed, which means the model cannot adjust it based on the data.

5.2 Parametric ReLU (PReLU)

PReLU extends Leaky ReLU by making the negative slope learnable:

$$f(x) = \begin{cases} x, & x > 0 \\ ax, & x \leq 0 \end{cases}$$

where a is learned during training.

What surprised me here is that such a small change — making α trainable — can improve performance. It gives the network more flexibility because each neuron can adapt differently. However, this also adds slightly more complexity and parameters to the model.

5.3 Exponential Linear Unit (ELU)

ELU uses an exponential function for negative inputs:

$$f(x) = \begin{cases} -x, & x > 0 \\ \alpha(e^{x-1}), & x \leq 0 \end{cases}$$

ELU stood out to me because it does not just "fix" ReLU, but changes its behavior more smoothly. The negative outputs are not just small — they are curved. This helps bring the average activation closer to zero, which can make learning more stable. Compared to ReLU, it feels like a more refined version, although it is slightly more expensive to compute.

5.4 GELU

GELU introduces a smooth, probabilistic behavior.

$$GELU(x) = 0.5x(1 + \tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)))$$

This function felt quite different from ReLU when I first saw it. Instead of making a sharp decision (zero or x), it gradually allows values through. This makes it more flexible, especially in modern architectures like transformers. However, it is also more complex mathematically.

5.5 Swish

Swish is defined as:

$$f(x) = x \cdot \sigma(x)$$

where the sigmoid function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Where $\sigma(x)$ is sigmoid.

Swish was interesting because it combines linear and non-linear behavior. It allows small negative values instead of removing them completely. From a practical point of view, it feels like a smoother version of ReLU that avoids its harsh cut-off at zero. However, it requires more computation due to the sigmoid function.

6. Python Implementation and Visualisation

To understand these functions more clearly, I implemented them in Python using NumPy and Matplotlib. The code focuses on three main visual ideas: the positive region of ReLU, the negative region where the dying ReLU problem appears, and a comparison between ReLU and the improved activation functions.

The positive-region graph shows why ReLU is attractive: for positive inputs, the function behaves like a straight line and keeps useful information unchanged. The negative-region graph shows the main weakness: all negative inputs are mapped to zero, which explains why neurons may stop learning. The comparison graph then makes it easy to see how Leaky ReLU, PReLU, ELU, GELU, and Swish change the negative side to keep some form of gradient or smoother behaviour.

These visualisations are important because they turn the formulas into something easier to interpret. A small change in the shape of the curve can have a major effect on optimisation, gradient flow, and overall model performance.

7. Discussion

One of the most useful lessons from studying activation functions is that there is no single best choice for every problem. ReLU remains a very strong default because it is simple, fast, and effective in many standard neural-network architectures. For many image classification and feedforward tasks, it still performs very well.

However, if a model shows signs of inactive neurons or unstable optimisation, improved functions may be better choices. Leaky ReLU and PReLU are practical when the main concern is preventing dead neurons. ELU is useful when smoother negative behaviour and more balanced activations are preferred. GELU and Swish are often chosen in more advanced deep-learning systems where smoothness and flexibility can provide a performance advantage.

This shows that activation functions are not just small technical details. They directly influence how well a network learns, how quickly it trains, and how stable the optimisation process becomes.

8. Conclusion

ReLU is one of the most important activation functions in machine learning because it is simple, computationally efficient, and highly effective. Its formula, $f(x) = max(0, x)$, is easy to implement, yet it made training deep neural networks much more practical. At the same time, its biggest weakness is the dying ReLU problem, where neurons may stop learning if they remain in the negative region.

Improved activation functions were created to address this issue. Leaky ReLU introduces a small negative slope, PReLU learns that slope automatically, ELU uses an exponential negative region, and GELU and Swish provide smoother alternatives. Together, these functions show that even a small mathematical change can significantly affect the behaviour and performance of a neural network.

Overall, studying ReLU and its variants gives a clearer understanding of why activation functions matter so much in deep learning. They are not just formulas placed between layers; they play a central role in whether a model learns well or struggles during training.

One thing I personally found interesting is how small mathematical changes in activation functions can significantly change how a neural network learns.

9. References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. Proceedings of ICML 2010.
- Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV 2015.
- Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2016). Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs).
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for Activation Functions.